Ex. No. 8	**Firestore Data Retrieval**
Date of Exercise	
Github Code Link (Keep as public)	
Video Link (Google Drive Link)	

Aim

To design and implement a Flutter application that retrieves information from Google Cloud Firestore based on user search input and displays the corresponding records dynamically.

Description

Google Cloud Firestore is a NoSQL, document-oriented cloud database. The cloud_firestore plugin lets a Flutter app query collections of documents directly from the client. A common pattern for "search as you type" style name lookups in Firestore is a prefix range query: ordering the collection by the target field and bounding it with startAt(query) and endAt('$query\uf8ff'), where \uf8ff is a very high Unicode code point that sorts after virtually any normal character, so every document whose field starts with the typed prefix falls inside the range.

This experiment implements a People Search screen backed by a people collection in Firestore. A PeopleRepository service wraps the query (ordered by name, bounded with startAt/endAt, limited to 20 results, with a request timeout so a slow or unreachable backend fails fast instead of hanging). The SearchScreen holds a TextField bound to the query, triggers the repository call when the user submits, and renders the results in a ListView of PersonTile cards. Three explicit UI states are handled: the initial "type a query" prompt, a loading spinner while the query is in flight, and an error/empty state. Because this submission does not have a live Firebase project attached (firebase_options.dart ships with placeholder credentials and a comment telling the student to run flutterfire configure), the app detects the placeholder configuration before even calling Firebase.initializeApp and shows a clear 'Firebase not configured' banner instead of crashing or hanging — once a real project is connected the exact same search code will return live documents.

Program

lib/models/person.dart

```
class Person {
  final String id;
  final String name;
  final String department;
  final String email;
```

```
  const Person({
    required this.id,
    required this.name,
    required this.department,
    required this.email,
  });

  factory Person.fromMap(String id, Map<String, dynamic> map) {
    return Person(
      id: id,
      name: (map['name'] ?? '') as String,
      department: (map['department'] ?? '') as String,
      email: (map['email'] ?? '') as String,
    );
  }
}
```

lib/services/people_repository.dart

```
import 'package:cloud_firestore/cloud_firestore.dart';

import '../models/person.dart';

/// Thin wrapper around the `people` Firestore collection used for the
/// prefix-search demo. Any Firestore error (e.g. no project configured)
/// is surfaced to the caller instead of crashing the UI.
class PeopleRepository {
  // A getter (not a field initializer) so FirebaseFirestore.instance is only
  // touched when a search actually runs, not when this class is constructed.
  CollectionReference<Map<String, dynamic>> get _collection =>
      FirebaseFirestore.instance.collection('people');

  Future<List<Person>> searchByName(String query) async {
    final trimmed = query.trim();
    if (trimmed.isEmpty) return [];

    final snapshot = await _collection
        .orderBy('name')
        .startAt([trimmed])
        .endAt(['$trimmed␣'])
        .limit(20)
        .get()
        .timeout(const Duration(seconds: 8));

    return snapshot.docs.map((doc) => Person.fromMap(doc.id,
doc.data())).toList();
  }
}
```

lib/widgets/firebase_status_banner.dart

```
import 'package:flutter/material.dart';

/// Shown whenever Firebase.initializeApp() failed (no project configured
/// yet) so the UI still renders instead of crashing.
class FirebaseStatusBanner extends StatelessWidget {
  const FirebaseStatusBanner({super.key});

  @override
  Widget build(BuildContext context) {
    return Container(
      width: double.infinity,
      color: Colors.amber.shade100,
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 10),
      child: Row(
        children: const [
          Icon(Icons.warning_amber_rounded, color: Colors.orange),
          SizedBox(width: 10),
          Expanded(
```

```
            child: Text(
              'Firebase not configured — run `flutterfire configure` with your
'
              'own project to enable live Firestore data.',
              style: TextStyle(fontSize: 12.5),
            ),
          ),
        ],
      ),
    );
  }
}
```

lib/widgets/person_tile.dart

```
import 'package:flutter/material.dart';

import '../models/person.dart';

class PersonTile extends StatelessWidget {
  final Person person;

  const PersonTile({super.key, required this.person});

  @override
  Widget build(BuildContext context) {
    return Card(
      margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
      child: ListTile(
        leading: CircleAvatar(child: Text(person.name.isNotEmpty ?
person.name[0] : '?')),
        title: Text(person.name),
        subtitle: Text('${person.department} • ${person.email}'),
      ),
    );
  }
}
```

lib/screens/search_screen.dart

```
import 'package:flutter/material.dart';

import '../models/person.dart';
import '../services/people_repository.dart';
import '../widgets/firebase_status_banner.dart';
import '../widgets/person_tile.dart';

class SearchScreen extends StatefulWidget {
  final bool firebaseReady;

  const SearchScreen({super.key, required this.firebaseReady});

  @override
  State<SearchScreen> createState() => _SearchScreenState();
}

class _SearchScreenState extends State<SearchScreen> {
  final _repository = PeopleRepository();
  final _controller = TextEditingController();

  List<Person> _results = [];
  bool _loading = false;
  String? _error;
  bool _searched = false;

  Future<void> _runSearch() async {
    if (!widget.firebaseReady) {
      setState(() {
        _searched = true;
        _results = [];
```

```
        _error = null;
      });
      return;
    }
    setState(() {
      _loading = true;
      _error = null;
      _searched = true;
    });
    try {
      final results = await _repository.searchByName(_controller.text);
      setState(() => _results = results);
    } catch (e) {
      setState(() => _error = 'Search failed: $e');
    } finally {
      setState(() => _loading = false);
    }
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Firestore Search')),
      body: Column(
        children: [
          if (!widget.firebaseReady) const FirebaseStatusBanner(),
          Padding(
            padding: const EdgeInsets.all(16),
            child: TextField(
              controller: _controller,
              textInputAction: TextInputAction.search,
              onSubmitted: (_) => _runSearch(),
              decoration: InputDecoration(
                labelText: 'Search by name',
                hintText: 'e.g. Arun',
                prefixIcon: const Icon(Icons.search),
                border: OutlineInputBorder(borderRadius:
BorderRadius.circular(12)),
                suffixIcon: IconButton(
                  icon: const Icon(Icons.arrow_forward),
                  onPressed: _runSearch,
                ),
              ),
            ),
          ),
          if (_loading) const Padding(
            padding: EdgeInsets.all(12),
            child: CircularProgressIndicator(),
          ),
          if (_error != null)
            Padding(
              padding: const EdgeInsets.symmetric(horizontal: 16),
              child: Text(_error!, style: const TextStyle(color: Colors.red)),
            ),
          Expanded(
            child: _buildResultsArea(),
          ),
        ],
      ),
    );
  }

  Widget _buildResultsArea() {
    if (!_searched) {
      return const Center(
        child: Text('Type a name and press search.', style: TextStyle(color:
```

```
Colors.grey)),
      );
    }
    if (!widget.firebaseReady) {
      return const Center(
        child: Padding(
          padding: EdgeInsets.all(24),
          child: Text(
            'No results — Firestore is not connected yet.\n'
            'Run flutterfire configure, add sample documents to the\n'
            '"people" collection, then search again.',
            textAlign: TextAlign.center,
            style: TextStyle(color: Colors.grey),
          ),
        ),
      );
    }
    if (_results.isEmpty && !_loading) {
      return const Center(child: Text('No matching records found.'));
    }
    return ListView.builder(
      itemCount: _results.length,
      itemBuilder: (context, index) => PersonTile(person: _results[index]),
    );
  }
}
```

lib/firebase_options.dart

```
import 'package:firebase_core/firebase_core.dart' show FirebaseOptions;
import 'package:flutter/foundation.dart' show defaultTargetPlatform, kIsWeb,
TargetPlatform;

/// Placeholder Firebase configuration.
///
/// Replace this file by running `flutterfire configure` from the project
/// root with your own Firebase project selected. That command regenerates
/// this file with real API keys and app IDs for every configured platform.
class DefaultFirebaseOptions {
  static FirebaseOptions get currentPlatform {
    if (kIsWeb) return web;
    switch (defaultTargetPlatform) {
      case TargetPlatform.android:
        return android;
      default:
        throw UnsupportedError(
          'DefaultFirebaseOptions have not been configured for this platform. '
          'Run `flutterfire configure` to generate real options.',
        );
    }
  }

  static const web = FirebaseOptions(
    apiKey: 'REPLACE_WITH_YOUR_API_KEY',
    appId: 'REPLACE_WITH_YOUR_APP_ID',
    messagingSenderId: 'REPLACE_WITH_YOUR_SENDER_ID',
    projectId: 'REPLACE_WITH_YOUR_PROJECT_ID',
    authDomain: 'REPLACE_WITH_YOUR_PROJECT_ID.firebaseapp.com',
    storageBucket: 'REPLACE_WITH_YOUR_PROJECT_ID.appspot.com',
  );

  static const android = FirebaseOptions(
    apiKey: 'REPLACE_WITH_YOUR_API_KEY',
    appId: 'REPLACE_WITH_YOUR_APP_ID',
    messagingSenderId: 'REPLACE_WITH_YOUR_SENDER_ID',
    projectId: 'REPLACE_WITH_YOUR_PROJECT_ID',
    storageBucket: 'REPLACE_WITH_YOUR_PROJECT_ID.appspot.com',
  );
}
```

lib/main.dart

```
import 'package:firebase_core/firebase_core.dart';
import 'package:flutter/material.dart';

import 'firebase_options.dart';
import 'screens/search_screen.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  var firebaseReady = false;
  final options = DefaultFirebaseOptions.currentPlatform;
  if (!options.apiKey.startsWith('REPLACE_WITH')) {
    try {
      await Firebase.initializeApp(options: options);
      firebaseReady = true;
    } catch (_) {
      firebaseReady = false;
    }
  }
  runApp(FirestoreSearchApp(firebaseReady: firebaseReady));
}

class FirestoreSearchApp extends StatelessWidget {
  final bool firebaseReady;

  const FirestoreSearchApp({super.key, required this.firebaseReady});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Firestore Search',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo),
        useMaterial3: true,
      ),
      home: SearchScreen(firebaseReady: firebaseReady),
    );
  }
}
```

Output Screenshots

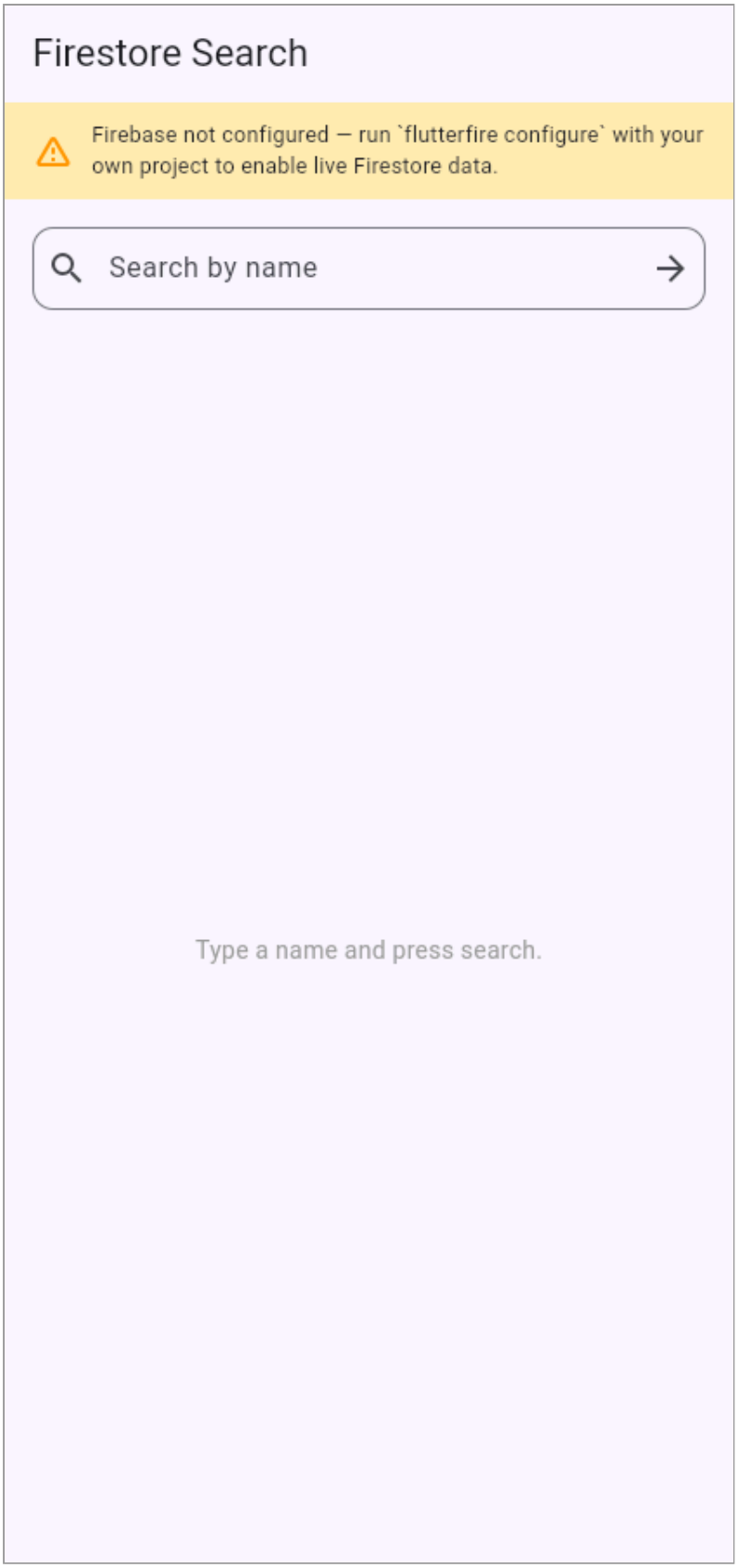


Fig 1. Search screen on first load, showing the 'Firebase not configured' banner (no live project attached yet).

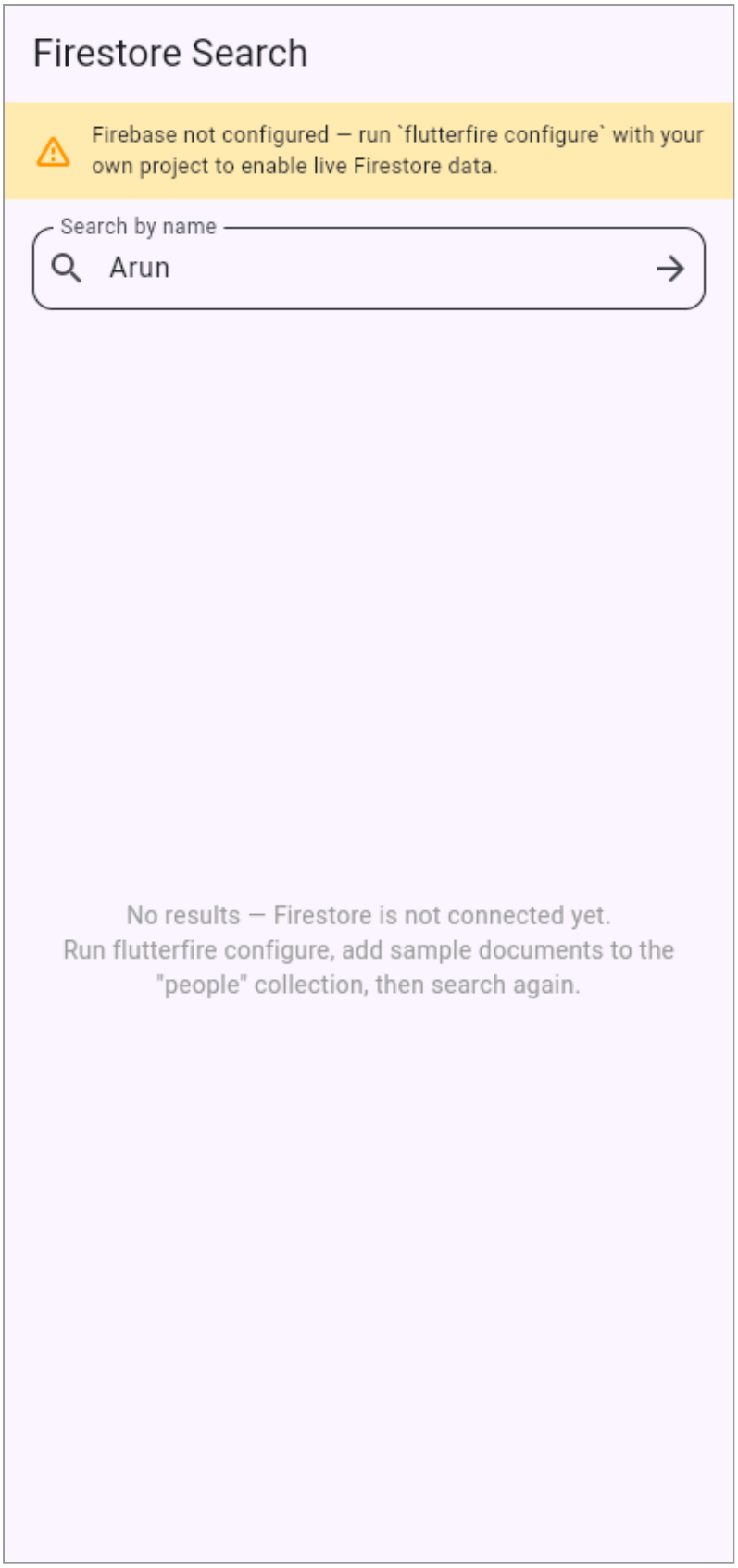


Fig 2. Submitting a search query ('Arun') with the graceful not-connected/no-results state. With a real Firebase project (via flutterfire configure) and documents in the 'people' collection, this same query path returns live matching records.

Result

The Flutter application was successfully designed to retrieve information from Google Cloud Firestore based on user search input, using a prefix range query (startAt/endAt) on the 'people' collection. The search UI, loading state and graceful not-connected/empty state were all verified in the browser. Live record retrieval will be verified once the student's own Firebase project is connected via flutterfire configure.